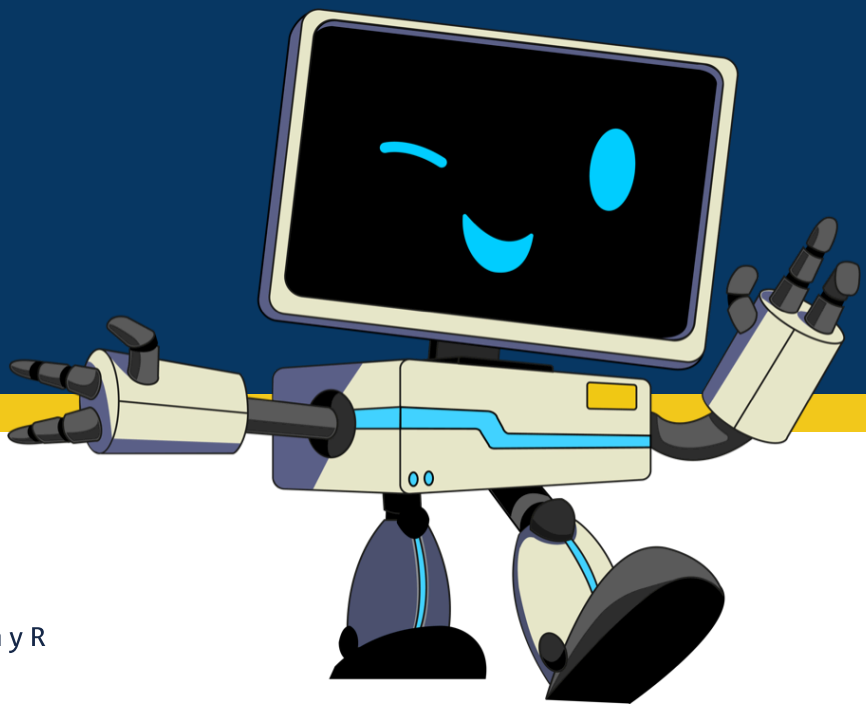
GUÍA DE REPASO · SESIÓN 2

Tomar decisiones: condicionales y lógica

Definiciones, ejemplos y ejercicios para repasar lo que vimos en clase.



CURSO LIBRE

Introducción a la Programación en Python y R

SEMESTRE

2026-2

ESTUDIANTE AUXILIAR

Yony Sebastián Chaparro Mesa

MATERIAL DEL CURSO

Moodle FCE

UIFCE
UNIDAD DE INFORMÁTICA

Unidad de Informática · Facultad de Ciencias Económicas
Universidad Nacional de Colombia, Sede Bogotá
uniic_bog@unal.edu.co

Cómo usar esta guía

Esta guía resume la segunda sesión del curso. Cada tema tiene **definiciones** (recuadros azules), **ejemplos de código** con su salida y **advertencias** sobre los errores más comunes (recuadros amarillos).

La mejor forma de repasar es con Google Colab abierto: **escribe tú mismo cada ejemplo**, ejecútalo y compara tu salida con la de la guía. Después cambia un valor y, antes de ejecutar, intenta adivinar qué camino tomará el programa.

TEMA	QUÉ DEBERÍAS PODER HACER AL TERMINAR
1. Comparaciones	Escribir comparaciones y predecir si dan `True` o `False`.
2. `if` y `else`	Hacer que un programa elija entre dos caminos.
3. Indentación	Saber qué instrucciones quedan dentro de un `if` y cuáles quedan fuera.
4. `elif`	Manejar más de dos opciones y ordenar bien las condiciones.
5. `and`, `or`, `not`	Combinar varias condiciones en una sola.
6. `input()` y decisiones	Pedir un dato, convertirlo a número y decidir con él.
7. Errores	Reconocer y corregir los errores más frecuentes de los condicionales.

Al final encontrarás un **resumen**, un **glosario**, **ejercicios** para practicar y sus **soluciones**.

1 Comparaciones

Hasta ahora, nuestros programas hacían siempre lo mismo, de arriba hacia abajo. Para que un programa pueda decidir, primero necesita hacerse preguntas que se respondan con sí o no.

> **DEFINICIÓN**
> Una **comparación** es una pregunta sobre dos valores. Su resultado siempre es un `bool`: `True` (verdadero) o `False` (falso).

```
nota = 3.5
print(nota >= 3.0)
print(nota == 5)
```

> **SALIDA**
> ```
> True
> False
> ```

Los seis operadores de comparación

SÍMBOLO	PREGUNTA	CON `nota = 3.5`	RESULTADO
`==`	¿Es igual?	`nota == 5`	**False**
`!=`	¿Es distinto?	`nota != 5`	**True**
`>`	¿Es mayor?	`nota > 4`	**False**
`<`	¿Es menor?	`nota < 4`	**True**
`>=`	¿Es mayor o igual?	`nota >= 3.0`	**True**
`<=`	¿Es menor o igual?	`nota <= 3.5`	**True**

- `>=` y `<=` se escriben en ese orden: `=>` no existe en Python.
- `True` y `False` llevan mayúscula inicial y no van entre comillas.

`=` guarda, `==` compara

Es la confusión más común de esta sesión. Un solo `=` **guarda** un valor en una variable. Dos signos `==` **preguntan** si dos valores son iguales.

`=` **asigna**: "guarda 3.5 en nota"

```
nota = 3.5
print(nota)
```

SALIDA
```
3.5
```

`==` **pregunta**: "¿nota es igual a 3.5?"

```
nota = 3.5
print(nota == 3.5)
```

SALIDA
```
True
```

También se puede comparar texto

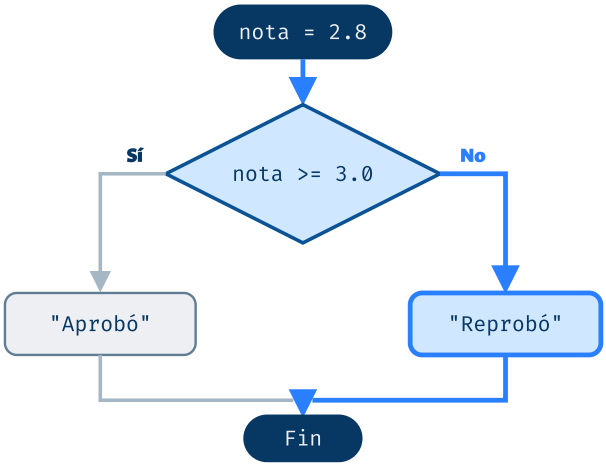
```
print("Ana" == "Ana")
print("Ana" == "ana")   # distingue mayúsculas y minúsculas
print("5" == 5)         # texto y número no son iguales
```

SALIDA
```
True
False
False
```

2 Tomar un camino u otro: `if` y `else`

> **DEFINICIÓN**
> Un **condicional** es una instrucción que ejecuta un bloque de código **solo si se cumple una condición**. Una **condición** es cualquier expresión que da `True` o `False`.

Con `if` ("si") y `else` ("si no") el programa elige entre dos caminos. Un **diagrama de flujo** dibuja esos caminos. El camino resaltado en azul es el que toma el programa con `nota = 2.8`:




```
nota = 2.8
if nota >= 3.0:
    print("Aprobó")
else:
    print("Reprobó")
```

> **SALIDA**
> ```
> Reprobó
> ```

Como 2.8 no es mayor o igual a 3.0, la condición da `False` y el programa toma el camino del `else`.

Anatomía de un `if`

1. La palabra `if`.
2. La **condición**: algo que da `True` o `False`.
3. Los **dos puntos** `:` al final de la línea. Se olvidan mucho.
4. La **indentación**: 4 espacios que marcan qué va dentro del `if`.

El `else` no lleva condición, pero sí lleva dos puntos. Además, es opcional: si no lo escribes y la condición da `False`, el programa simplemente sigue con lo que viene después.

```
nota = 4.0
if nota >= 3.0:
    print("Aprobó")
print("Fin")
```

> **SALIDA**
> ```
> Aprobó
> Fin
> ```

CONSEJO

Prueba el programa con `nota = 3.0` exacto. Con `>=` aprueba; con `>` reprueba. Un solo símbolo cambia el resultado, así que elige el operador con cuidado.

3 La indentación no es decoración

DEFINICIÓN

La **indentación** (o sangría) son los espacios al inicio de una línea. En Python **define la estructura del programa**: las líneas con la misma indentación debajo del `if` forman un **bloque**, y ese bloque solo se ejecuta si la condición se cumple.

Se usan 4 espacios. En Colab, la tecla **Tab** los inserta por ti, y después de escribir los dos puntos y presionar Enter, la indentación aparece sola.

¿Dentro o fuera?

Estos dos programas solo se diferencian en la indentación de la última línea, y no hacen lo mismo:

"Fin" dentro del `if`

```
nota = 2.0
if nota >= 3.0:
    print("Aprobó")
    print("Fin")
```

SALIDA

```
(no imprime nada)
```

"Fin" fuera del `if`

```
nota = 2.0
if nota >= 3.0:
    print("Aprobó")
print("Fin")
```

SALIDA

```
Fin
```

Ninguno da error. La condición es `False` en los dos casos, pero a la derecha `"Fin"` está fuera del bloque, así que se imprime siempre.

Si falta la indentación

```
nota = 3.5
if nota >= 3.0:
print("Aprobó")
```

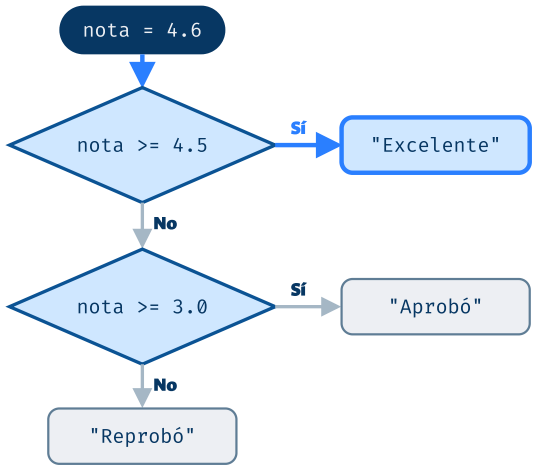
```
IndentationError: expected an indented block after 'if' statement on line 2
```

Python espera un bloque indentado después de los dos puntos, y no lo encuentra.

4 Varias opciones con `elif`

DEFINICIÓN

`elif` es la abreviatura de “else if” (“si no, entonces si...”). Permite revisar otra condición cuando la anterior no se cumplió. Python revisa las condiciones **en orden** y **se detiene en la primera verdadera**.




```
nota = 4.6
if nota >= 4.5:
    print("Excelente")
elif nota >= 3.0:
    print("Aprobó")
else:
    print("Reprobó")
```

SALIDA

```
Excelente
```

- Empieza con **un** `if`.
- Puede tener **tantos** `elif` como necesites, cada uno con su condición.
- Termina, si quieres, con **un solo** `else`, sin condición.
- Solo se ejecuta **un** camino.

Cómo lo recorre Python

Este es el recorrido del mismo programa con `nota = 3.4`:

PASO	LÍNEA	RESULTADO	QUÉ HACE PYTHON
1	`if nota >= 4.5:`	**False**	Salta su bloque y sigue a la siguiente condición.
2	`elif nota >= 3.0:`	**True**	Imprime `Aprobó` y se detiene.
3	`else:`	—	Ni siquiera lo revisa.

El orden importa

Con `nota = 4.6`, ¿qué imprime este programa?

```
nota = 4.6
if nota >= 3.0:
    print("Aprobó")
elif nota >= 4.5:
    print("Excelente")
```

SALIDA

```
Aprobó
```

OJO

Esto es un **error lógico**: no aparece ningún mensaje de error, pero el resultado está mal. La primera condición ya es verdadera, así que Python nunca llega a revisar el `elif`. Por eso las condiciones **más exigentes van primero**.

`elif` no es lo mismo que otro `if`

Con varios `if` seguidos, Python revisa **todos**, y puede ejecutarse más de un camino:

Con `elif`: un solo camino

```
nota = 4.6
if nota >= 4.5:
    print("Excelente")
elif nota >= 3.0:
    print("Aprobó")
```

SALIDA

```
Excelente
```

Con dos `if`: se revisan ambos

```
nota = 4.6
if nota >= 4.5:
    print("Excelente")
if nota >= 3.0:
    print("Aprobó")
```

SALIDA

```
Excelente
Aprobó
```

5 Combinar condiciones: `and`, `or`, `not`

DEFINICIÓN

Los **operadores lógicos** combinan o invierten condiciones. El resultado también es `True` o `False`.

OPERADOR	SE LEE	DA True CUANDO…	CON edad = 20 Y tiene_documento = False	RESULTADO
`and`	"y"	Las **dos** condiciones son verdaderas.	`edad >= 18 and tiene_documento`	**False**
`or`	"o"	**Al menos una** es verdadera.	`edad >= 18 or tiene_documento`	**True**
`not`	"no"	La condición es **falsa**: invierte el valor.	`not tiene_documento`	**True**

Ejemplo: ¿puede votar?

```
edad = 20
tiene_documento = True
if edad >= 18 and tiene_documento:
    print("Puede votar")
else:
    print("No puede votar")
```

SALIDA

```
Puede votar
```

Para votar se necesitan **las dos cosas**, por eso usamos `and`. Si usáramos `or`, alguien de 15 años con documento podría votar:

PERSONA	edad >= 18	tiene_documento	and	or
20 años, con documento	**True**	**True**	**True**	**True**
20 años, sin documento	**True**	**False**	**False**	**True**
15 años, con documento	**False**	**True**	**False**	**True**
15 años, sin documento	**False**	**False**	**False**	**False**

Invertir con `not`

```
tiene_documento = False
if not tiene_documento:
    print("Recuerda traer tu documento")
```

SALIDA

```
Recuerda traer tu documento
```

OJO

Cada lado de `and` y `or` debe ser una **condición completa**. `edad >= 18 and <= 25` da `SyntaxError`: se escribe `edad >= 18 and edad <= 25`. Y `dia == "sábado" or "domingo"` no da error, pero siempre se cumple: se escribe `dia == "sábado" or dia == "domingo"`.

6 Decidir con datos del usuario: `input()`

En la sesión 1 vimos que `input()` **siempre devuelve texto**. Para comparar con números, primero hay que convertir ese texto.

Sin convertir: error

```
edad = input("¿Cuántos años tienes? ")
if edad >= 18:
    print("Eres mayor de edad")
```

```
TypeError: '>=' not supported between instances of 'str' and 'int'
```

El mensaje dice que no se puede usar `>=` entre un texto (`str`) y un entero (`int`).

Convirtiendo: funciona

```
edad = int(input("¿Cuántos años tienes? "))
if edad >= 18:
    print("Eres mayor de edad")
```

SALIDA

```
¿Cuántos años tienes? 20
Eres mayor de edad
```

Usa `int()` para enteros y `float()` para decimales, como una nota de 3.5:

```
nota = float(input("¿Cuál es tu nota? "))
if nota >= 4.5:
    print("Excelente")
elif nota >= 3.0:
    print("Aprobó")
else:
    print("Reprobó")
```

SALIDA

```
¿Cuál es tu nota? 3.5
Aprobó
```

OJO

Si el usuario escribe `3.5` y usas `int(input(...))`, Python responde con `ValueError`, porque ese texto no es un entero. Para notas, usa `float()`.

7 Errores frecuentes

Recuerda lo que vimos en la sesión 1: empieza por la **última línea** del mensaje, que es donde está la pista principal.

ERROR Y QUÉ SIGNIFICA	CÓDIGO CON ERROR	CORRECCIÓN

`SyntaxError`
Faltan los dos puntos: `expected ':'`.
	`nota = 3.5`
	`if nota >= 3.0`
	`    print("Aprobó")`
		`nota = 3.5`
		`if nota >= 3.0:`
		`    print("Aprobó")`

`SyntaxError`
Usaste `=`, que guarda, en lugar de `==`, que compara.
	`nota = 3.5`
	`if nota = 3.5:`
	`    print("Igual")`
		`nota = 3.5`
		`if nota == 3.5:`
		`    print("Igual")`

`IndentationError`
Falta la sangría del bloque.
	`nota = 3.5`
	`if nota >= 3.0:`
	`print("Aprobó")`
		`nota = 3.5`
		`if nota >= 3.0:`
		`    print("Aprobó")`

`TypeError`
Se está comparando texto con un número.
	`edad = input("Edad: ")`
	`if edad >= 18:`
	`    print("Mayor")`
		`edad = int(input("Edad: "))`
		`if edad >= 18:`
		`    print("Mayor")`

Otros errores que vas a encontrar

- `else if nota >= 3.0:` → `SyntaxError`: en Python se escribe `elif`.
- `else nota < 3.0:` → `SyntaxError`: el `else` no lleva condición.
- `If nota >= 3.0:` → `SyntaxError`: Python distingue mayúsculas; se escribe `if`.
- `print(true)` → `NameError`: se escribe `True`, con mayúscula.
- **Sin mensaje de error:** condiciones en mal orden o `and` en lugar de `or`. El programa corre, pero el resultado está mal. Por eso conviene probarlo con varios valores.

Según la versión de Python, el texto de algunos mensajes puede cambiar un poco, pero el tipo de error es el mismo.

Tres ideas clave

1. Una condición siempre se evalúa como True o False.
2. if / elif / else permite que el programa tome caminos distintos.
3. En Python la indentación no es decoración: define la estructura.

Glosario

Bloque	Grupo de líneas con la misma indentación que se ejecutan juntas.
Booleano	Tipo de dato `bool`. Solo tiene dos valores: `True` y `False`.
Comparación	Pregunta sobre dos valores que da `True` o `False`, como `nota >= 3.0`.
Condición	Expresión que da `True` o `False` y decide qué camino toma el programa.
Condicional	Instrucción que ejecuta un bloque solo si se cumple una condición.
Convertir	Cambiar un dato de un tipo a otro, por ejemplo con `int()` o `float()`.
Diagrama de flujo	Dibujo de los pasos y los caminos que puede tomar un programa.
`elif`	"Si no, entonces si...". Revisa otra condición cuando la anterior no se cumplió.
`else`	"Si no". Se ejecuta cuando ninguna condición anterior se cumplió.
Error lógico	El programa corre sin mensajes de error, pero el resultado está mal.
`if`	"Si". Ejecuta su bloque cuando la condición es verdadera.
Indentación	Espacios al inicio de una línea. En Python define qué va dentro de cada bloque.
Operador de comparación	Símbolo que compara dos valores: `==`, `!=`, `>`, `<`, `>=`, `<=`.
Operador lógico	Palabra que combina o invierte condiciones: `and`, `or`, `not`.

Ejercicios

Resuélvelos en Colab antes de mirar las soluciones. Avanza hasta donde llegues.

Ponte a prueba: ¿qué imprime cada línea?

Escribe tu respuesta **antes** de ejecutar el código.

CÓDIGO	TU RESPUESTA
`print(5 == 5.0)`	
`print("5" == 5)`	
`print(3 > 3)`	
`print(3 >= 3)`	
`print(not 4 > 2)`	
`print(2 > 1 and 1 > 2)`	
`print(2 > 1 or 1 > 2)`	

¿Qué camino toma?

Sigue el programa con el dedo, línea por línea, y escribe lo que imprime.

CÓDIGO	TU RESPUESTA

`nota = 3.0`
`if nota > 3.0:`
`  print("Aprobó")`
`else:`
`  print("Reprobó")`	

`x = 7`
`if x > 5:`
`  print("A")`
`elif x > 3:`
`  print("B")`
`else:`
`  print("C")`	

`n = 4`
`if n % 2 == 0:`
`  print("Par")`
`print("Fin")`	

Encuentra el error

Di qué tipo de error produce cada programa y corrígelo.

CÓDIGO	ERROR Y CORRECCIÓN

`nota = 4.0`
`if nota >= 3.0`
`    print("Aprobó")`	

`nota = 4.0`
`if nota = 4.0:`
`    print("Cuatro")`	

`edad = 20`
`if edad >= 18:`
`print("Mayor")`	

`nota = 4.0`
`if nota >= 4.5:`
`    print("Excelente")`
`else if nota >= 3.0:`
`    print("Aprobó")`	

Ejercicios de la clase

BÁSICO **1.** Pide un número y di si es positivo, negativo o cero.

BÁSICO **2.** Pide la nota de un estudiante y muestra “Aprobó” o “Reprobó”.

INTERMEDIO **3.** Clasificador de IMC: pide peso y estatura, calcula `peso / estatura**2` y muestra la categoría. Usa las categorías de la OMS: bajo peso (menos de 18.5), normal (de 18.5 a menos de 25), sobrepeso (de 25 a menos de 30) y obesidad (30 o más).

INTERMEDIO **4.** Una tienda da 10 % de descuento si la compra supera $100.000 y 20 % si supera $200.000. Calcula el valor final.

RETO **5.** Pide un año y di si es bisiesto (divisible entre 4, excepto los divisibles entre 100 que no lo sean entre 400). Pista: usa `%`.

RETO **6.** Mini “piedra, papel o tijera” entre dos jugadores que escriben su elección.

> **CONSEJO**
> ¿Te atascaste? Dibuja primero el **diagrama de flujo** en papel y programa después. En el ejercicio 4, piensa qué condición debe ir primero.

Soluciones

Hay muchas formas correctas de resolver cada ejercicio. Si tu código es distinto pero da el resultado correcto, también está bien.

Ponte a prueba

CÓDIGO	SALIDA	POR QUÉ
`print(5 == 5.0)`	`True`	Tienen el mismo valor, aunque uno sea `int` y el otro `float`.
`print("5" == 5)`	`False`	Uno es texto y el otro es número.
`print(3 > 3)`	`False`	3 no es mayor que 3.
`print(3 >= 3)`	`True`	3 sí es igual a 3.
`print(not 4 > 2)`	`False`	`4 > 2` es `True` y `not` lo invierte.
`print(2 > 1 and 1 > 2)`	`False`	`and` necesita las dos, y la segunda es falsa.
`print(2 > 1 or 1 > 2)`	`True`	`or` necesita al menos una, y la primera es verdadera.

¿Qué camino toma?

CÓDIGO	SALIDA	POR QUÉ

`nota = 3.0`
`if nota > 3.0:`
`  print("Aprobó")`
`else:`
`  print("Reprobó")`
	`Reprobó`	Con `>`, 3.0 no cumple la condición. Con `>=` sí la cumpliría.

`x = 7`
`if x > 5:`
`  print("A")`
`elif x > 3:`
`  print("B")`
`else:`
`  print("C")`
	`A`	Las dos primeras condiciones son verdaderas, pero Python se detiene en la primera.

`n = 4`
`if n % 2 == 0:`
`  print("Par")`
`print("Fin")`
	`Par`
	`Fin`
		4 es par, y `"Fin"` está fuera del `if`: se imprime siempre.

Encuentra el error

CÓDIGO	ERROR	CORRECCIÓN

`nota = 4.0`
`if nota >= 3.0`
`    print("Aprobó")`
	`SyntaxError`: faltan los dos puntos.	`nota = 4.0`
		`if nota >= 3.0:`
		`    print("Aprobó")`

`nota = 4.0`
`if nota = 4.0:`
`    print("Cuatro")`
	`SyntaxError`: para comparar se usa `==`.	`nota = 4.0`
		`if nota == 4.0:`
		`    print("Cuatro")`

`edad = 20`
`if edad >= 18:`
`print("Mayor")`
	`IndentationError`: falta la sangría.	`edad = 20`
		`if edad >= 18:`
		`    print("Mayor")`

`nota = 4.0`
`if nota >= 4.5:`
`    print("Excelente")`
`else if nota >= 3.0:`
`    print("Aprobó")`
	`SyntaxError`: en Python se escribe `elif`.	`nota = 4.0`
		`if nota >= 4.5:`
		`    print("Excelente")`
		`elif nota >= 3.0:`
		`    print("Aprobó")`

Próxima sesión: listas, bucles y funciones. ¿Y si quisiéramos repetir algo cien veces? Trae tus dudas de esta guía a la clase o escríbelas en el Moodle FCE.